Digital Microelectronics II: Project Development Diary

DAOUDI–CARACCI Milane

01/06/2025

Aalto University
School of Electrical
Engineering

Introduction

This document presents the design and implementation of a PIC microcontroller using VHDL. The project focuses on creating a functional replica of the PIC16F84A architecture, including its core components:

- Arithmetic Logic Unit (ALU) implementation with full instruction support
- Memory subsystem with synchronous/asynchronous access modes
- Instruction decoder with four-state finite state machine
- Comprehensive testbenches for verification

Chapter 1 will mainly focus on the design of each unit. I will explain the way I thought when designing each unit, my design choices and how I adapted to simulation results in order to meet specifications given or that I established.
Chapter 2 will focus on the top simulation. I will demonstrate that I successfully implemented a PIC microcontroller able to read instructions from an hexfile. I will generate hexfiles with piklab and assembly code.
Chapter 3 will deal with synthesis.

Contents

1 Entities

1.1 ALU

1.1.1 Entity Design

Designing the ALU presented several challenges, which I systematically resolved through iterative debugging and VHDL best practices. I chose to first define a package that defines all the operations code. You need to be careful and not select opcode like AND. Instead, I chose AND2. In this package there is also a function that take a 6 characters string and convert it to type alu_operation. I used this function in the testbench to read from an input file. This solution wasn't the best, I should have used comas but it worked and I didn't have to use this method again in the future. That's why I didn't changed it for something better.

```
1  package ALU_Types is
2      type ALU_operation is (ADD, AND2, COMF, DECF, INCF, IOR, NOP,
           RLF,
3                             RRF, SUB, SWAPF, XOR2, CLRF, CLRW, MOVF,
4                             MOVWF, MOVLW, BCF, BSF);
5
6      -- Function to convert string to ALU_operation
7      function string_to_ALU_operation(op_str : string) return ALU_
           operation;
8  end ALU_Types;
9
10 -- Package body where the function is defined
11 package body ALU_Types is
12
13     function string_to_ALU_operation(op_str : string) return ALU_
           operation is
14     begin
15         -- Check the string and map it to the corresponding ALU_
               operation value
16         if op_str = "ADD   " then
17             return ADD;
18         elsif op_str = "AND2  " then
19             return AND2;
20         elsif op_str = "COMF  " then
21             return COMF;
22         elsif op_str = "DECF  " then
23             return DECF;
```

Listing 1.1: ALU Operations Package Definition

- **Problem 1: Pipeline Delay in ALU Output**

  Initial Issue: The ALU initially introduced a one-cycle delay between receiving inputs and producing the output. However, the design required combinational logic (immediate output upon input change).

  Root Cause: Misunderstanding of VHDL's process sensitivity list and signal assignment behavior. My initial approach incorrectly introduced a register-like delay:

```
process(clk)
begin
    result <= A + B; -- Intermediate signal
    alu_result <= result; -- Introduced 1-cycle delay
end process;
```

Listing 1.2: Example

  Solution: Removed the intermediate result signal and assigned alu_result directly within a combinational process. I also instantiated 2 N bits ripple carry adder (one for addition, one for substraction) in a way that they always output a result.

```
ADDER_INSTANCE: Nbit_ripple_carry_adder
    port map (A  => A, B  => B, S  => adder_result, CO => adder
       _CO);

B_2complement <= std_logic_vector(unsigned(not B) +1);

SUBSTRACTOR_INSTANCE : Nbit_ripple_carry_adder
    port map (A  => A, B => B_2complement, S  => sub_result, CO
        => sub_CO);
```

Listing 1.3: Components instantiations

```
--process for taking care of result and Z_flag
process(A, B, operation, bit_select, status_in, adder_result,
   sub_result, clear_mask)
begin
    case operation is
        when ADD =>
            result <= adder_result;
            update_Zflag <= '1';

        when AND2 =>
            result <= A and B;
            update_Zflag <= '1';

        when COMF =>
            result <= not A;
            update_Zflag <= '1';
```

Listing 1.4: Beginning of first process

- **Problem 2: Status Flag Management**

  Initial Issue: The status_out port was driven by multiple processes, leading to a multi-driver conflict.

  Solution: Restructured the logic into two dedicated processes: Process 1 takes care of result and update_Zflag and the second process takes care of status flag.

```
1  --process for taking care of status
2  process(result, update_Zflag, status_in, operation, adder_CO,
       sub_CO)
3  begin
4      status <= status_in;
5      if update_Zflag = '1' then
6          if result = "00000000" then
7              status(2) <= '1';  -- Set Z flag if result is zero
8          else
9              status(2) <= '0';  -- Clear Z flag if result is non
                   -zero
10         end if;
11     end if;
12
13     if operation = ADD then
14         status(0) <= adder_CO;
15         status(1) <= A(3) and B(3); --carry_out_4bit
16     elsif operation = SUB then
17         status(0) <= sub_CO;
18         status(1) <= A(3) and B_2complement(3);
19     elsif operation = RLF then
20         status(0) <= A(7);
21     elsif operation = RRF then
22         status(0) <= A(0);
23     elsif operation = NOP then
24         status <= "000";
25     end if;
26 end process;
```

Listing 1.5: Second process for status flag

- **Problem 3: Bit Manipulation (BSF/BCF)**

  Initial Issue: Assigning clear_mask inside a process caused unexpected delays.

  Solution: Defined clear_mask combinationally outside the process:

```
1  clear_mask <=
2  (others => '1') when operation /= BCF and operation /= BSF else
       -- Default case
3  (0 => '0', others => '1') when operation = BCF and bit_select =
       "000" else
4  (1 => '0', others => '1') when operation = BCF and bit_select =
       "001" else
5  (2 => '0', others => '1') when operation = BCF and bit_select =
       "010" else
6  (3 => '0', others => '1') when operation = BCF and bit_select =
       "011" else
```

```
7  (4 => '0', others => '1') when operation = BCF and bit_select =
       "100" else
8  (5 => '0', others => '1') when operation = BCF and bit_select =
       "101" else
9  (6 => '0', others => '1') when operation = BCF and bit_select =
       "110" else
10 (7 => '0', others => '1') when operation = BCF and bit_select =
       "111" else
11 (others => '1') when operation = BCF else  -- Default for BCF
12 (0 => '1', others => '0') when operation = BSF and bit_select =
       "000" else
13 (1 => '1', others => '0') when operation = BSF and bit_select =
       "001" else
14 (2 => '1', others => '0') when operation = BSF and bit_select =
       "010" else
15 (3 => '1', others => '0') when operation = BSF and bit_select =
       "011" else
16 (4 => '1', others => '0') when operation = BSF and bit_select =
       "100" else
17 (5 => '1', others => '0') when operation = BSF and bit_select =
       "101" else
18 (6 => '1', others => '0') when operation = BSF and bit_select =
       "110" else
19 (7 => '1', others => '0') when operation = BSF and bit_select =
       "111" else
20 (others => '0') when operation = BSF else  -- Default for BSF
21 (others => '1');  -- Fallback default
```

Listing 1.6: Continuous assignment of clear mask

1.1.2 Testbench and simulation

The module was easy to connect to the top level entity. I think that defining a package to define a new type called alu_operation (ADD, AND2, ETC...) was a good idea because it made my input program clearer:
ADD 01100100 00000111 000 000
ADD 11111111 00000001 000 000
AND2 11111111 00001111 000 000
AND2 00000000 11111111 000 000
Yet as a counter part it was a bit harder to read from the file input_program.txt. I had to make sure there was exactly 6 characters space included before the first operand. I could used a coma to make the reading easier but I succeeded like this.

1.2 Memory Implementation

1.2.1 Entity Design

The memory implementation presented interesting architectural choices. While conceptually straightforward, the design required careful consideration of access methodologies:

- **Memory Access Strategy**: I initially implemented a **synchronous read/write** design with dual ports (`read_addr` and `write_addr`) and separate data registers (`mem_data_in` and `mem_data_out`). This approach worked correctly but was later modified to use **asynchronous/combinational reads** to better align with the decoder's timing requirements. The final implementation drives `data_out` immediately when `read_en` is asserted.

```
1 -- Combinational read
2 data_out <= mem(to_integer(unsigned(read_addr))) when read_en
     ='1'
3             else (others => '0');
```

Listing 1.7: Continuous assignment of data_out enabling combinational reading

- **Status Register Handling**: The special case of the status register at address `0x03` required particular attention. The design prioritizes top-level `data_in` over `status_in` during writes to this address. Additionally, I introduced a `status_write_enable` signal to properly handle bit-set (BSF) and bit-clear (BCF) operations. This refinement emerged during debugging, demonstrating the iterative nature of hardware design and verification.

```
1 elsif (status_write_enable) then --for RLF and RRF
2     mem(to_integer(unsigned(STATUS_ADDR))) <=
3         (DATA_WIDTH-1 downto 3 => '0') & mem_status_in;
4     report "RLF instruction here";
5 end if;
```

Listing 1.8: Use of status_write_enable

1.2.2 Testbench Implementation

The testbench development focused on practical verification. First, I wrote a testbench for the memory and verified simple read and write instructions like these ones:

```
1  -- Test 1: Simple write then read
2  report "Test 1: Write then read from different addresses";
3  write_addr <= x"10";
4  data_in <= x"AA";
5  mem_status_in <= "101";
6  write_en <= '1';
7  wait for clk_period;
8  write_en <= '0';
9
10 read_addr <= x"10";
11 read_en <= '1';
12 wait for clk_period;
```

```
13 assert data_out = x"AA" report "Test 1 failed: Data mismatch"
       severity error;
14 assert mem_status_out = "101" report "Test 1 failed: Status
       mismatch" severity error;
15 read_en <= '0';
16 wait for clk_period;
```

Listing 1.9: Testbench testing only the memory

Then, I study the relation between ALU and memory and implemented a testbench that instantiates alu and memory and connect them such as:

- **ALU-Memory Interface**: Established clear signal relationships:
  - `mem_status_in` ← `alu_status_out`
  - `alu_status_in` ← `mem_status_out`
- **Testing Methodology**: Implemented direct test vectors rather than file I/O for simplicity and debuggability:

```
1  -- Test 1: Write value to memory
2  report "Writing 0x55 to address 0x10";
3  mem_write_addr <= x"10";
4  mem_data_in <= x"55";
5  mem_write_en <= '1';
6  wait for CLK_PERIOD;
7  mem_write_en <= '0';
8  -- wait for CLK_PERIOD;
9
10 -- Test 2: Read value back from memory
11 report "Reading from address 0x10";
12 mem_read_addr <= x"10";
13 mem_read_en <= '1';
14 wait for CLK_PERIOD;
15 mem_read_en <= '0';
16 -- wait for CLK_PERIOD;
17
18 -- Test 3: Perform ADD operation (5 + 3)
19 report "Testing ADD operation (5 + 3)";
20 -- Write first operand to memory
21 mem_write_addr <= x"20";
22 mem_data_in <= x"05";
23 mem_write_en <= '1';
24 wait for CLK_PERIOD;
25 mem_write_en <= '0';
26 -- wait for CLK_PERIOD;
```

Listing 1.10: Testbench combining ALU and memory

The test sequence included:

1. Basic write/read verification
2. ALU operations with memory operands
3. Result storage back to memory

4. Comprehensive status flag testing

This approach provided excellent debugging visibility while maintaining test coverage.

1.3 Decoder

To implement the decoder, I started reading the datasheet thoroughly. Before that, I only read it for the ALU to check what operations I needed to implement. I understood how to execute each instruction using the four different states: `iFetch`, `Mread`, `Execute` and `Mwrite`. I understood how instructions were thought: most of the instructions have an opcode so we can identify the operation, an address or a literal coded with a certain number of bits. Thanks to this study, I drew a simple finite state machine which is not complete (so I won't submit it here). I just used it for myself in order to have clearer ideas on how to start coding.
At first, I thought the decoder didn't need several outputs/inputs except `clk`, `rst`, `instruction` and `pc`. But then, when I first simulated my decoder I thought I couldn't access internal signals but I just forgot that GTKWave enables you to do that by clicking on the module in the GTKWave graphic interface. I remembered this only after implementing a decoder with more outputs and inputs, but this is not important because choosing what are the outputs of the decoder is a "choice" and since we don't have any criteria to respect, I decided to keep it like that. All signals used by submodules memory and ALU are also outputs of the decoder. That way, it's also easy for me to select these signals when launching GTKWave because I don't have to go through the submodules.

1.3.1 Instruction Package

The first thing I decided to do before implementing the decoder is a package named `instruction_pkg` that contains a new type definition: `instruction_type`. I use this type to make the difference between:

- Byte-oriented
- Bit-oriented
- Literal and control (`lANDc`)
- Special operations (`CALL`, `GOTO`)

In this package I also define a procedure called `data_extraction` that:

- Assigns to `operation` the type of operation executed by the instruction (given as an input of the procedure)
- Extracts the rest of the data from each instruction

```
1 package instruction_pkg is
2     -- Instruction type enumeration
3     type instruction_type is (byte_oriented, bit_oriented, lANDc,
          CLRW, special, unknown);
4
5     -- Constants for special instructions
```

```
6       constant NOP_INSTRUCTION : std_logic_vector(13 downto 0) := "
            00000000000000";
7       constant RETURN_INSTRUCTION : std_logic_vector(13 downto 0) :=
            "00000000001000";
8       constant ADDR_WIDTH : integer := 7;
9       constant DATA_WIDTH : integer := 8;
10
11      -- Data extraction procedure
12      procedure data_extraction (
13          -- Input
14          signal instruction : in std_logic_vector(13 downto 0);
15          -- Global output
16          signal operation : out instruction_type;
17          -- Output byte oriented
18          signal opcode_byte : out std_logic_vector(5 downto 0); -- 6
                bits
19          signal d_byte : out std_logic;
20          signal f_byte : out std_logic_vector(6 downto 0);
21          -- Output bit-oriented
22          signal opcode_bit : out std_logic_vector(3 downto 0); -- 4
                bits
23          signal b_bit : out std_logic_vector(2 downto 0);
24          signal f_bit : out std_logic_vector(6 downto 0);
25          -- Output literal and control
26          signal opcode_lANDc : out std_logic_vector(5 downto 0); --
                6 bits
27          signal k_literal : out std_logic_vector(7 downto 0);
28          -- Output special
29          signal opcode_special : out std_logic_vector(2 downto 0);
30          signal k_special : out std_logic_vector(10 downto 0);
31          -- Memory interface
32          signal mem_read_address : out std_logic_vector(6 downto 0)
33      );
34  end package instruction_pkg;
```

Listing 1.11: Beginning of instruction package

For instance, byte-oriented operations are recognized because they all start by "00". We then assign:

- The opcode to `opcode_byte`
- The direction to `d_byte`
- The 7-bit address to `mem_read_address` (used during reading)

I call this procedure in `iFetch` and it makes my code easier to read. The process of extracting data is purely combinational, so the procedure is the right way to do this.
At first, this package also defined a type `state` which could be `iFetch`, `Mread`, `Execute` or `Mwrite`. But then when I first simulated the decoder, I couldn't see the signals `current_state` and `next_state` because GTKWave doesn't know how to read this new type. I should have used strings: `"iFetch"`, etc. But I finally decided to use `std_logic_vector`:

- "00" for `iFetch`

- "01" for `Mread`
- ...

1.3.2 Module Connections

In `decoder.vhd`, I instantiate ALU and memory using positional assignment and signals I used as output of my decoder. They are connected together this way:

- `alu_status_out_reg` is just a register that retains the value of `alu_status_out` when calculated. We assign `alu_status_out_reg` to `mem_status_in` so the memory takes care of writing this status to address `0x03h`. This is taken care of in the memory implementation and detailed above.
- We then connect `mem_status_out` and `alu_status_in`. Anytime, the memory is assigning a value to `mem_status_out` which is the value it found at address `0x03h` and we use this as input for the ALU.

This part was a bit tricky. At first I connected `mem_status_in` and `alu_status_out` but I saw in the testbench that it wasn't the good solution because when not used, the ALU would always assign 0 for `alu_status_out` (it wasn't retaining its value). So I used a register to do this.
I will speak of `status_write_enable` later (last signal of memory outputs).

1.3.3 State Machine Implementation

I first implemented a synchronous process with asynchronous reset. I may change this functionality later. This process takes care of:

- Assigning `next_state` to `current_state` each rising clock edge
- Updating `pc` when the current state is `Execute` (so that `pc` can be changed exactly at the end of the instruction)

At first, `pc` was always incremented but then I added an if statement that assigned a different value than `pc + 1` to `pc` if the operation was `CALL`, `GOTO`, `RETLW` or `RETURN`. This other value was `pc_reg` and is assigned in the other process (I will speak about it later). This first process is very simple.
The second process takes care of the logic and describes a case statement which reacts according to `current_state` and calculates `next_state`. I will speak about things that I find important in this process or troubles I encountered and how I resolved them.
At first, I wanted to reset all outputs of the decoder but I figured that I only need to reset:

- The memory contents (nothing to do because the memory is already linked with the top-module reset)
- `next_state` because I had a problem when simulating it (I was blocked at state "01" after a reset which is not normal)
- The accumulation register `w_register`

State Machine Details

The first part of case statement is for fetching the instruction. It:

- Calls the extraction procedure
- Determines if the instruction needs reading or not
- Chooses the `next_state` which can be `Mread` or `Execute` based on instruction
- If the `next_state` is reading, it will also assert `mem_read_en` which enables reading from memory address `mem_read_addr`

The second part of the case is for memory reading only. For operations that need reading from memory, it will keep the value read by assigning `mem_data_out` to `operand` which then will be used to feed the ALU. The next state is always `Execute` here.
The next part of case statement is for `Execute` state. Based on operation (`byte_oriented`, `bit_oriented`, `lANDc`, `special`...) and opcode (`opcode_byte`, `opcode_bit`, `opcode_lANDc`, `opcode_special`...), it determines how to feed the ALU properly using `operand`, `w_register`, `k_literal` or `k_special`...
The last part of case statement is `Mwrite`. In this part:

- I assign `alu_status_out` to `alu_status_out_reg` for the reasons I explained earlier
- If operation is `BSF` or `BCF`, I assert `status_write_enable` that enables these operations to rotate properly through carry bit

I did this because when I simulated an `RRF` instruction it wasn't properly doing the operation wanted, so I decided to change the implementation of my memory by adding a `status_enable` signal as input. When it is activated and `mem_write_enable` isn't asserted, it still writes the content of `status_in` to `STATUS_ADDR 0x03h`. Doing multiple simulations and testbenches helped me a lot to observe different errors and their causes and also to fix problems by trying different solutions.
The `next_state` when memory writing is always `iFetch`.
An if statement is then taking care of writing `alu_result` to the proper address or `w_register` depending on `d` (if 1, result is written to address else to `w_register`).

1.3.4 Testing and Debugging

With the testbench, I simulated ALL instructions one by one, including `CALL`, `GOTO`, `RETLW`, `RETURN`.
A problem arose when trying the instruction `ADDWF`. It was the first time I was reading an operand from the memory. The problem was that I was receiving the operand one cycle after asking for it (after asserting `mem_read_en`). I decided to implement combinational read because before this I was using a synchronous read. I just changed a bit my memory implementation and it worked.
I also had many problems with status when doing `RLF` and `RRF` instructions but I think I came to a good solution with `status_write_enable` and `status_out_reg` as I explained earlier. At first I was connecting `alu_status_out` to `mem_status_in` but the fact that `alu_status_out` was "000" except when the ALU was used provoked incorrect writing to status address `0x03h` in memory. That's when I decided to connect `alu_status_out_reg` to `mem_status_in` instead of `alu_status_out`.

Once you read correctly from memory, write correctly to it and take care of status then most of operations worked. I just had to code `call`, `goto`, `retlw` and `return`.
In order to code them, I would assign `pc_reg` in `Execute` depending on the operation asked. `pc_reg` would then keep its value and I would assign `pc_reg` to `pc` in first process when doing `call`, `goto`, `retlw` and `return`. I chose to do this because you can't drive `pc` with 2 different processes.
Simulations took most of my time when doing this because I wanted to do a complete testbench to not have problems later.

1.4 Stack 8×13

I designed a simple stack with 8 levels of 13 bits each. Push and pop operations cannot be asserted simultaneously as this functionality was not required. The stack pointer is neither directly readable nor writable, and the output always reflects the top of the stack. This last feature allows me to read directly from the stack without enabling any signal, as it's combinational.
Following a TA's advice, I found existing stack implementation code online since many stack designs already exist. I modified a few parameters to match my specific requirements (number of levels, bit width, etc.).
For testing, I performed the following operations:

- Pushing and popping a few elements
- Pushing until the stack was full
- Popping until it was empty

After testing multiple cases, I gained a thorough understanding of how the stack operates. Debugging the code helped me comprehend many aspects of its functionality. The implementation worked correctly without any problems.

1.5 Connection Between Stack and Rest of the Logic

I modified my decoder implementation to connect the stack properly. The stack handles push and pop operations for specific instructions:

- **CALL**: Pushes `pc+1` to the stack by placing the value on `stack_data_in` and asserting `push`
- **RETLW** and **RETURN**: Retrieves the top of stack (`stack_data_out`) to feed `pc_reg`, then pops the stack by asserting `pop`

The decoder implementation was updated by:

- Removing the `tos` (top of stack) signal
- Adding new decoder outputs: `push`, `pop`, `stack_data_in`, and `stack_data_out`
- Implementing the required control logic

These changes were incorporated into a new VHDL file named `pic_rtl.vhdl`.

1.5.1 Stack Limitations

The 8-level stack allows for 8 nested subprogram calls. However, calling a 9th subprogram would overwrite the first return address, causing program errors. This represents a fundamental limitation of the stack depth.

1.5.2 Verification

To verify the program counter-stack connection:

- Adapted the testbench from Exercise 6
- Created a test case with:
  - A `CALL` to a subprogram executing `MOVLW` and `ADDLW`
  - Return to `pc+1` (pushed during `CALL`) using `RETLW`
  - Additional tests for `RETURN` instruction
- All test cases executed correctly

Conclusion

At this stage of development, all individual components have been rigorously verified through independent unit testing. The first phase of RTL design has been successfully completed with the top-level module `pic_rtl` capable of executing all specified instructions from the datasheet (excluding those explicitly excluded from the project requirements). All implemented instructions execute correctly within 3 or 4 clock cycles without errors.
A critical design consideration involves the timing of instruction assignment: new instructions must be assigned to the `instruction` signal of `pic_rtl` exclusively on the falling edge of the clock signal. This architectural decision was made after observing that rising-edge assignment introduced stability issues not present during falling-edge assignment. Importantly, this timing constraint:

- Does not affect instruction execution semantics
- Maintains the same instruction throughput
- Preserves the timing relationship between consecutive instructions

2 Top Simulation

2.1 Piklab

Beginning with Piklab, I first validated the development environment using the provided `led_blinker.asm` code. My prior experience with microprocessor programming from the "Computers and Microprocessors" course enabled rapid verification of this basic functionality.
To systematically verify the microprocessor implementation, I adapted my original decoder testbench approach to assembly language. The testbench evaluated instruction groups sequentially, with hardware resets between test segments. Since assembly lacks direct hardware reset capability, I implemented this testing framework through three separate projects:

- `testbench1.piklab` (First instruction group)
- `testbench2.piklab` (Second instruction group)
- `testbench3.piklab` (Third instruction group)

During initial testing, development was hindered by compilation failures in the provided `hex_file_reader` utility. This toolchain limitation prevented both successful builds and debugging of the assembly testbenches, requiring alternative verification methods.

2.2 Final Testbench Implementation

After resolving the toolchain issues, I successfully implemented the testbench using the corrected `hex_file_reader` utility. The testbench instantiates `pic_rtl` along with all associated signals.

2.2.1 Program Memory Loading Mechanism

The instruction loading process utilizes the `hex_file_reader` package, specifically the `read_ihex_file` procedure. This procedure requires:

- Input: The file path of assembly code generated by Piklab (via "Build Project")
- Output: An array containing the program memory (instructions in hexadecimal format)

Initial implementation attempts directly connected a `program_memory` signal as the procedure output. However, compilation failed because:

- The procedure requires a `variable` type output
- Conversion attempts to `variable` and `shared variable` types proved unsuccessful

The final solution implements:

- A temporary variable `temp_mem` within a wrapper procedure `load_program_memory`
- Proper signal driving from the temporary variable to the top-level `program_memory`

```
1  -- Procedure to load memory (avoiding shared variable issues)
2    procedure load_program_memory(
3        file_name : in string;
4        signal mem : out program_array) is
5        variable temp_mem : program_array := (others => (others =>
             '0'));
6    begin
7        read_ihex_file(file_name, temp_mem);
8        mem <= temp_mem;
9    end procedure;
```

Listing 2.1: Load program memory procedure

2.2.2 Testbench Control Flow

The stimulus process executes the following sequence:

- Calls `load_program_memory` to initialize instructions
- Deactivates the reset signal
- Waits for 100 clock periods
- Asserts `simulation_done` to force the clock signal to 0

```
1  stimulus: process
2        constant hex_file_path : string := "/home/mdaoudi/Documents
             /ELEC-E9540/PIC16F84A/piklab/testbench3.hex";
3    begin
4        -- Load HEX file into program memory
5        load_program_memory(hex_file_path, program_memory);
6
7        -- Release reset after 2 clock cycles
8        wait for CLK_PERIOD * 2.5;
9        reset <= '0';
10
11       -- Let the simulation run for a while
12       wait for CLK_PERIOD * 100;
13
14       -- End simulation
15       simulation_done <= true;
16       wait;
17   end process;
```

Listing 2.2: Stimulus process loading program memory

2.2.3 Instruction Fetch Implementation

The instruction fetch mechanism operates by:

- Converting the program counter value `pc` to integer `pc_int` on each falling edge
- Using `pc_int` to index the `program_memory` array

```
1   instruction_fetch: process(clk)
2         variable pc_int : integer;
3     begin
4         if falling_edge(clk) and reset = '0' then
5             -- Convert PC to integer for array indexing
6             pc_int := to_integer(unsigned(pc));
7
8             -- Fetch instruction from program memory
9             if pc_int < inst_mem_size then
10                instruction <= program_memory(pc_int);
11            else
12                -- Handle PC out of bounds (optional)
13                instruction <= (others => '0');
14                report "PC out of bounds: " & integer'image(pc_int)
                      severity warning;
15            end if;
16        end if;
17    end process;
```

Listing 2.3: Instruction Fetch Process

2.2.4 Note

Before going through the different testbenches, I just wanted to say that I forgot to change in the makefile the vcd format. I should have used the fst format. That's why I don't show alu_op on further simulation results because I was using .vcd format.

2.2.5 TESTBENCH 1: BYTE-ORIENTED INSTRUCTIONS

The first testbench was about testing all byte-oriented instructions. I didn't do a complicated assembly program, it is very simple but tests all instructions. It doesn't test all cases, only random cases that I chose because I don't think we need to test all cases. I will explain how instructions succeed one by one.

```
;*****Set up the Constants****

STATUS  equ 03h      ;Address of the STATUS register
TRISA   equ 85h      ;Address of the tristate register for port A
PORTA   equ 05h      ;Address of Port A
COUNT1  equ 08h      ;First adress
COUNT2  equ 09h      ;Second counter for our delay loops


;**** First part of code****
Start  movlw    03h     ;value 3 to 0x08
       movwf    COUNT1  ;transfer
       movlw    0Eh     ;value 14 to W
       addwf    COUNT1, w  ; add count1(3) and w(14) and put it in W
       movwf    COUNT1     ;move W(17) to COUNT1
       andwf    COUNT1, w   ;and between W=17 and COUNT1=17
       comf     COUNT1, f ;complement of COUNT1 and move it to W
       decf     COUNT1, w   ; decrement COUNT1 and move it to W
       incf     COUNT2, f  ; increment COUNT2 and move it to COUNT2
       iorwf   COUNT2,  w  ;  or between W and COUNT2
       clrf    COUNT1 ; empty COUNT1
       movf   COUNT1, w ; verify COUNT 1 is empty
       movf  COUNT2, w; move count2 to W
       clrw   ;


;****End of the program****

end                 ;Needed by some compilers,
                    ;and also just in case we miss
                    ;the goto instruction.
```

Figure 2.1: Assembly code

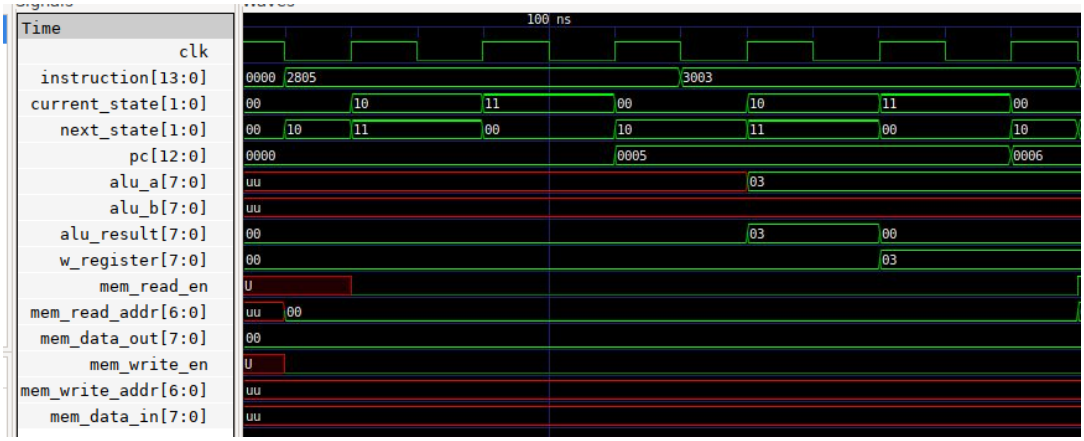

Figure 2.2: GOTO and MOVLW

- **initial go to**
  The first instruction is always go to main located at 05h in program memory. It charges program counter with the correct value so the next instruction can be assigned correctly.

- **movlw 03h**
  Movlw puts 03h in alu_A when the state is Execute and then assign alu_result which is 03h to w_register on state Mwrite (this is not a write to memory of course).

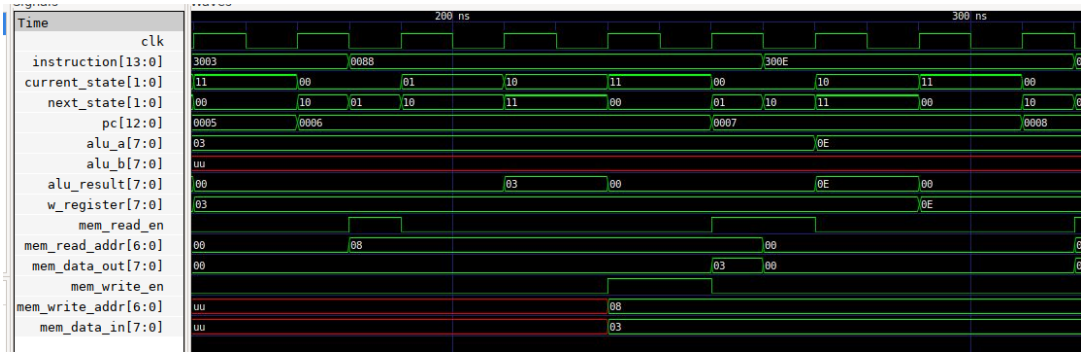

Figure 2.3: MOVWF and MOVLW

- **movwf COUNT1**
  Movwf puts 03h in alu_A and alu_result on Execute state. On Mwrite, it assigns mem_data_in with alu_result (it is the value we want to write), mem_read_addr with the address of COUNT1 which is 08h and also asserts mem_write_enable.

- **movlw 0Eh**
  It assigns 0Eh to w_register.

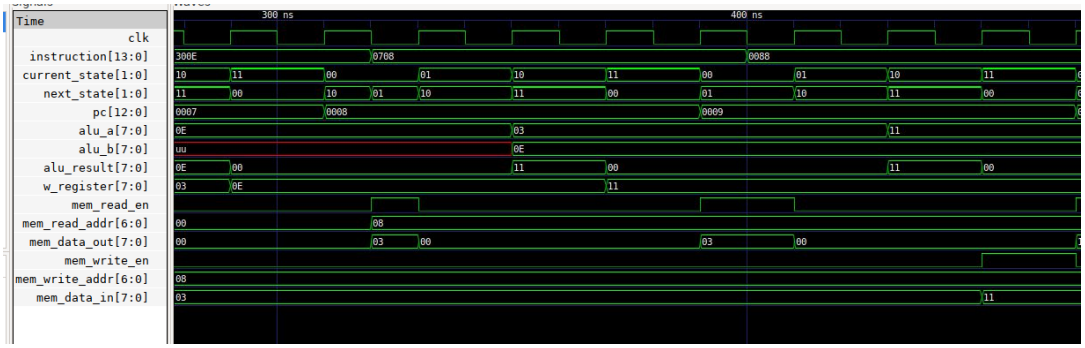

Figure 2.4: ADDWF and MOVWF

- **addwf COUNT1, w**
  It first reads the value by asserting mem_read_en and assigning COUNT1 address to mem_read_addr. We can see that the value mem_data_out is 03h at this moment. It should be this value because we wrote 03h to 08h address before. Then, you can see it feeds the ALU with the value written 03h and the value of w_register 0Eh. The result 11h is good and put to w_register.

- **movwf COUNT1**
  It moves 11h to address 08h by utilizing mem_write_en, mem_data_in and mem_write_addr.

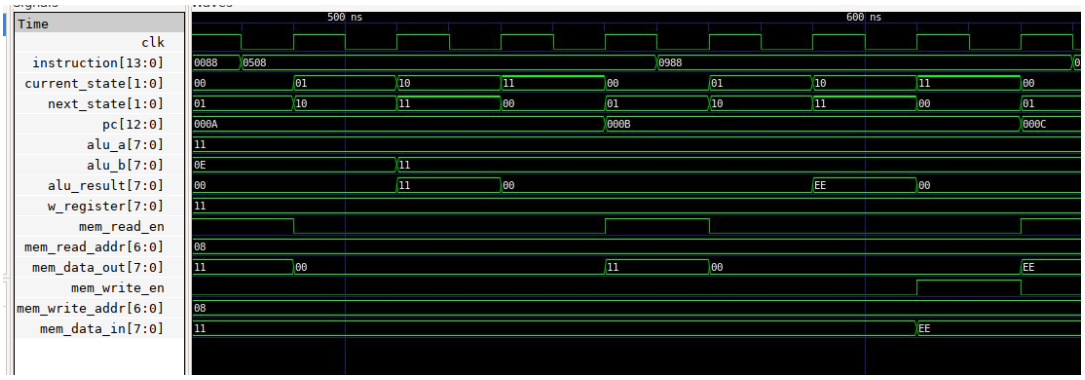

Figure 2.5: ANDWF and COMF

- **andwf COUNT1, w**
  It first reads the value from COUNT1 address 08h which is 11h and then feeds this value to alu_A and the w_register value 11h to alu_B. The result of 11h AND 11h is 11h. w_register keeps the same value.

- **comf COUNT1, f**
  It first reads the value from COUNT1 address 08h, feeds alu_A. Alu_result becomes the complement of 11h which is EEh and then it writes this value back to address 08h.

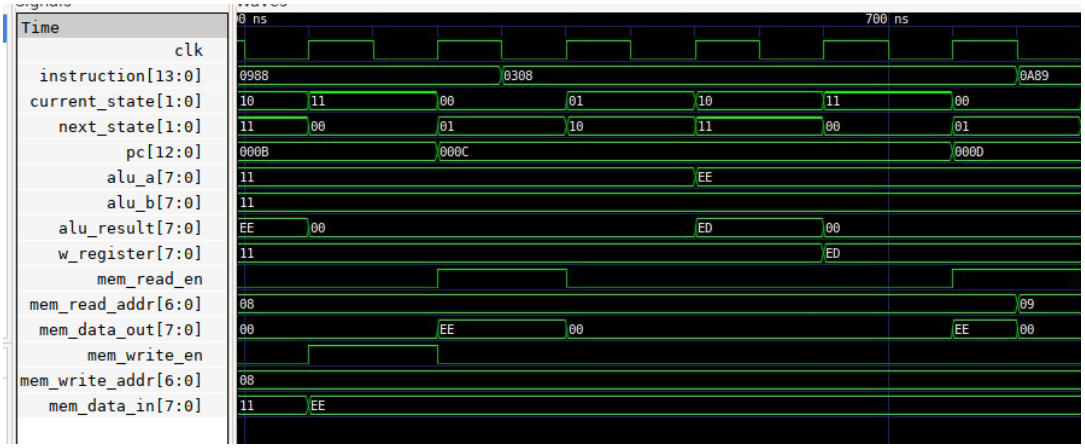

Figure 2.6: DECF

- **decf COUNT1, w**
  It first reads the value from COUNT1 address 08h, feeds alu_A. Alu_result becomes EEh - 01h which is EDh. This value is then assigned to w_register.

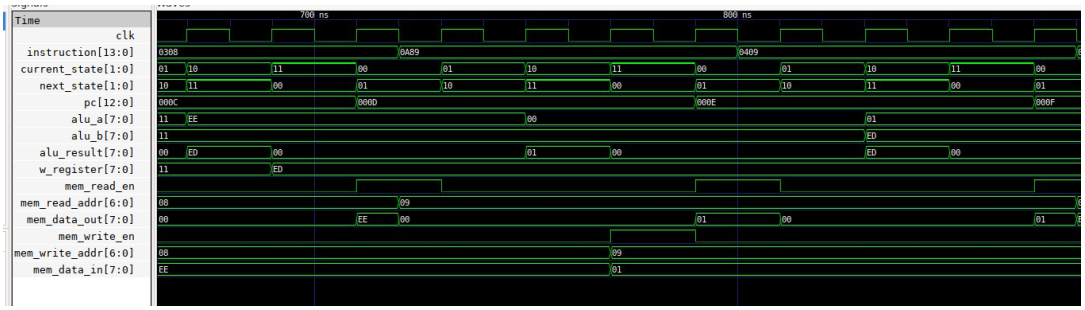

Figure 2.7: INCF and IORWF

- **incf COUNT2, f**
  It first reads the memory content of 09h which is 0 and then feeds alu_A with this value. The ALU increments it and outputs 01h on alu_result. This value is then written back to 09h address.

- **iorwf COUNT2, w**
  It reads the value 01h from COUNT2 address and feeds alu_A with 01h. It also feeds alu_B with w_register which is EDh. It then performs inclusive OR between the 2. The result EDh is good. It then writes the value to w_register which is still EDh.

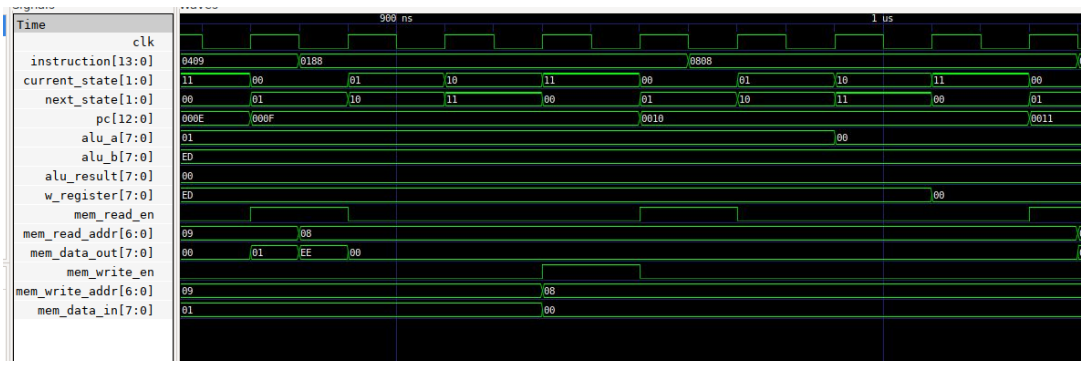

Figure 2.8: CLRF and MOVF

- **clrf COUNT1**
  Clrf just writes 00h to COUNT1 memory address.

- **movf COUNT1, w**
  This enabled me to verify that COUNT1 address was cleared correctly.

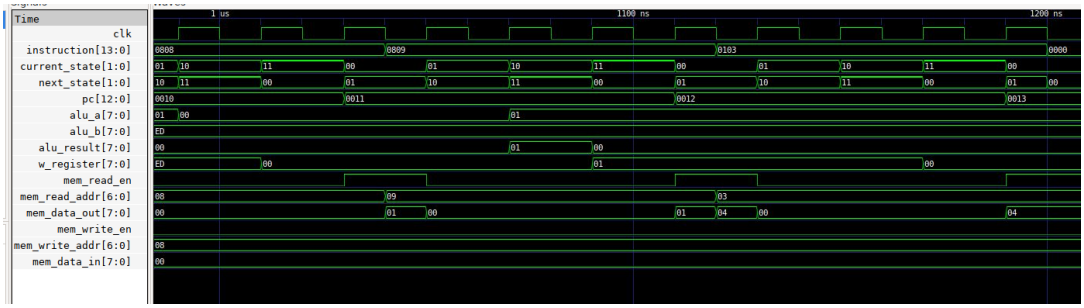

Figure 2.9: MOVF and CLRW

- **movf COUNT2, w**
- **clrw**
  Puts 0 in w correctly.

Every operation succeeds. The only problem of my design is that when we assign a new instruction that doesn't need reading after an instruction that needed reading, a reading operation is still processed. This is due to the fact that the next instruction is always assigned on falling edges. Thus after the end of an instruction that needed reading, the process goes back to "00" iFetch on a rising edge (and starts reading because it's still the same instruction) then the instruction changes and the process goes to "00" state on a falling edge. To change this, I have to change and assign new instructions on rising edge but as I said earlier I would have to change my decoder implementation which I don't want to do. Moreover, reading is not important. It doesn't change the memory contents. This will maybe cost some more power consumption but that's all.

2.2.6 TESTBENCH 2: REST OF BYTE-ORIENTED AND BIT-ORIENTED INSTRUCTIONS

```
RESET CODE 0x000
     goto main

ISR CODE 0x004
     goto isr_handler

MAIN CODE
isr_handler  ; interrupt code goes here

main

; your code goes here

;*****Set up the Constants****

STATUS  equ 03h      ;Address of the STATUS register
TRISA   equ 85h      ;Address of the tristate register for port A
PORTA   equ 05h      ;Address of Port A
COUNT1  equ 08h      ;First adress
COUNT2  equ 09h      ;Second counter for our delay loops


;**** First part of code****
Start incf STATUS, f ;
      movlw 31h ;
      movwf COUNT1 ;
      rlf COUNT1, w;
      rrf COUNT1, w;
      subwf COUNT1, w;
      swapf COUNT1, w;
      xorwf COUNT1, w;
      bsf COUNT2, 2;
      bcf COUNT2, 2;


;****End of the program****

end                 ;Needed by some compilers,
                    ;and also just in case we miss
                    ;the goto instruction.
```

Figure 2.10: Assembly code

The first instruction is the initial goto 05h.

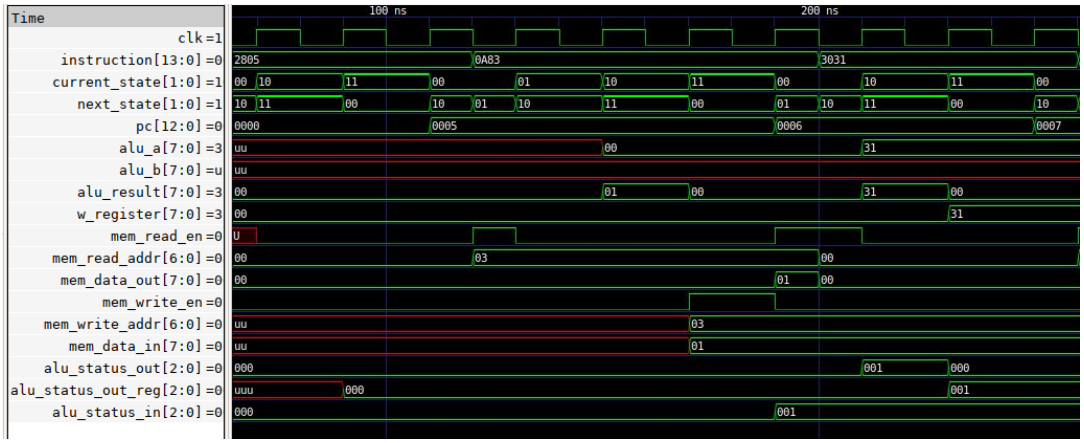

Figure 2.11: INCF and MOVLW operations

- **incf STATUS, f**: The second instruction was chosen to prepare for a rlf opera-

tion. To create an observable effect, I set the carry status bit by incrementing the STATUS register located at 0x03h in memory. During the Mwrite stage, we write 01h to memory address 0x03h. One cycle after writing this, alu_status_in (which is also mem_status_out) becomes "001" as expected. One cycle later, the ALU processes the next instruction while maintaining alu_status_out as "001" since the carry status bit remains unchanged. When an instruction doesn't modify status bits, alu_status_out equals alu_status_in. The alu_status_out_reg retains its value and connects to mem_status_in to prevent overwriting the STATUS register in memory.

- **movlw 31h** correctly moves 31h to w_register, and **movwf COUNT1** successfully moves 31h to memory location 0x08h. This value will be used for both RLF and RRF operations.

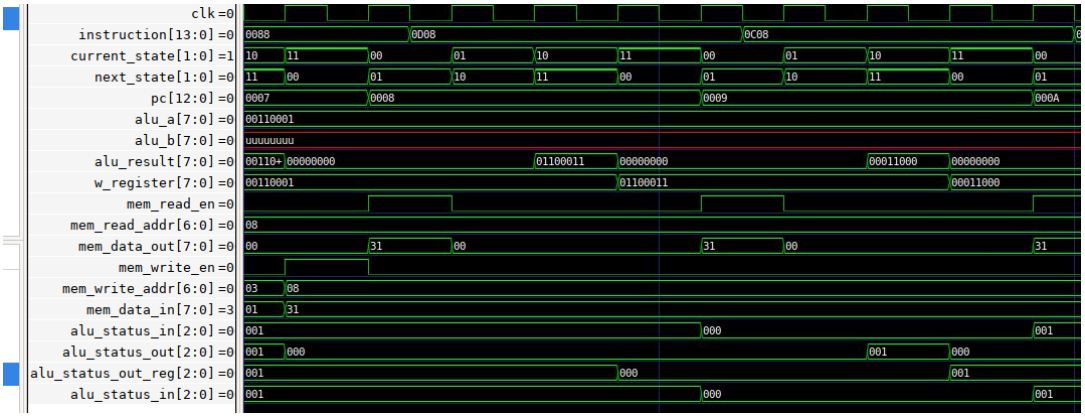

Figure 2.12: RLF and RRF operations

- **rlf COUNT1, w**: The instruction first reads memory location 0x08h and correctly retrieves 31h ("00110001" in binary). During the Execute stage, alu_A rotates left through the carry bit. With carry initially set to 1, the operation produces the correct result "01100011". We observe that alu_status_out_reg becomes "000" one cycle after the Execute state. This value propagates to mem_status_in, causing the memory to update the STATUS register at 0x03h. The update is confirmed when alu_status_in (mem_status_out) changes from "001" to "000". The final result is properly stored in w_register.

- **rrf COUNT1, w**: This instruction processes the same 31h value ("00110001"). With the carry bit now 0, the right rotation produces "00011000" and sets alu_status_out to "001". One cycle later, alu_status_out_reg updates to "001" and feeds into memory, which successfully updates the STATUS register as confirmed by alu_status_in becoming "001".

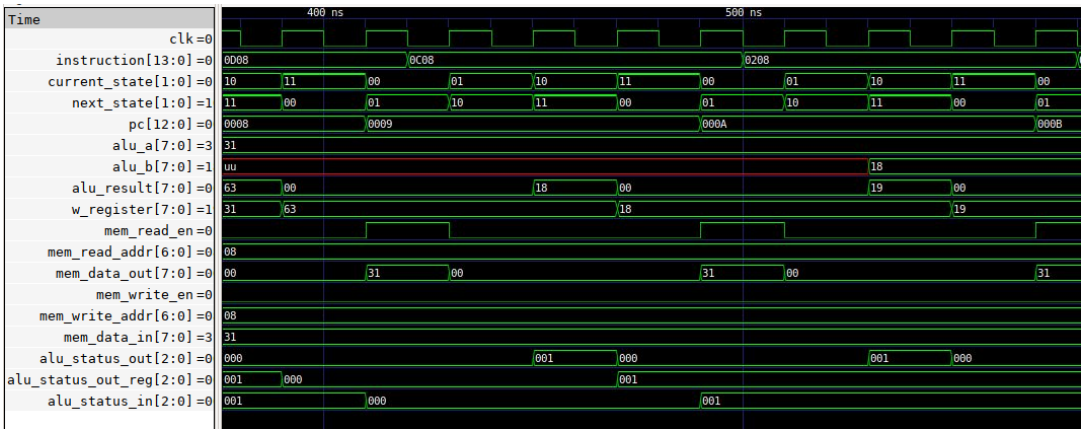

Figure 2.13: RRF and SUBWF operations

- **subwf COUNT1, w**: The instruction properly reads from memory location 0x08h, feeding alu_A with 31h and alu_B with the w_register value. The subtraction 31h - 19h correctly yields 19h, which is written to w_register.

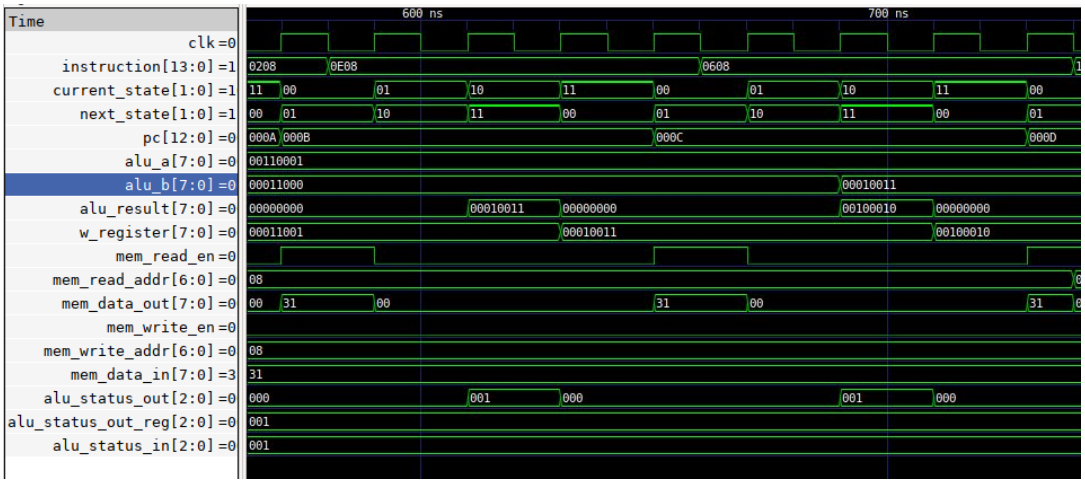

Figure 2.14: SWAPF and XORWF operations

- **swapf COUNT1, w**: The instruction reads the value correctly and performs the nibble swap operation, transforming "00110001" to "00010011". The result is accurately written to w_register.
- **xorwf**: Executed with "00110001" (from memory) and "00010011" (from w_register), the XOR operation produces the correct result "00100010".

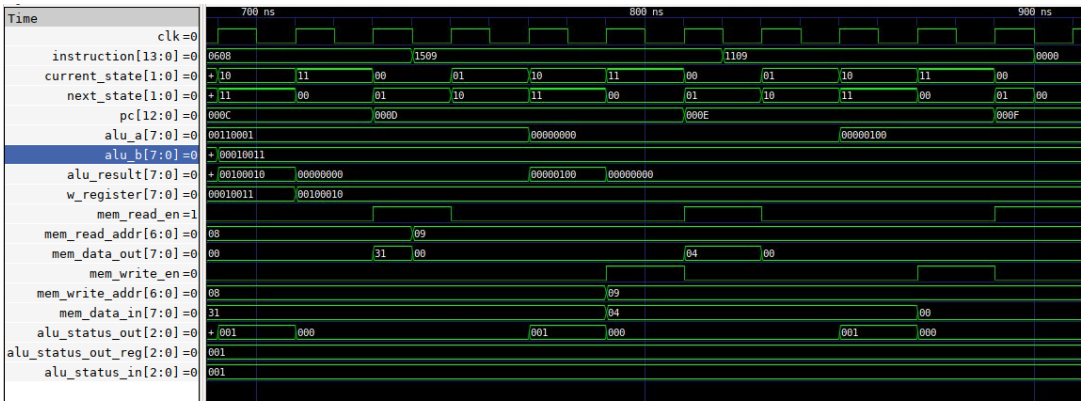

Figure 2.15: BSF and BCF operations

- **bsf COUNT2, 2**: This instruction operates on memory location 0x09h, which remained untouched until this point. It reads 0, sets the third bit (position 2), resulting in "00000100" (04h), and writes this back to 0x09h. I forgot to add alu_bit_select signal on gtk wave but you can see that the result is correct.
- **bcf COUNT2, 2**: The instruction reads the recently written value "00000100", clears the same bit, and writes back 00h to 0x09h. The sequence of bsf followed by bcf effectively leaves the memory location unchanged, demonstrating both bit operations work correctly.

2.2.7 TESTBENCH 3: PROGRAM CONTROL INSTRUCTIONS

```
RESET CODE 0x000
    goto    main        ; Jump to main program

ISR CODE 0x004
    goto    isr_handler ; Interrupt vector

MAIN CODE
isr_handler ; interrupt code goes here
;***** Constants *****
STATUS  equ 03h         ; STATUS register
TRISA   equ 85h         ; TRIS A register
PORTA   equ 05h         ; PORT A
COUNT1  equ 08h         ; Counter 1
COUNT2  equ 09h         ; Counter 2

;***** Main Program *****
main

Start:
    ; Demonstrate movlw and addlw
    movlw   0x10        ; Load 0x10 into W
    addlw   0x20        ; Add 0x20 to W (W now 0x30)
    movwf   COUNT1       ; Output result to COUNT1

    ; Call a subroutine
    call    math_operations  ; Call math subroutine

    ; Demonstrate goto
    goto    main        ; Infinite loop

;***** Subroutines *****
math_operations:
    iorlw   0xF0        ; OR operation
    sublw  0xFA;
    retlw   0xFF        ; Return with value 0xFF in W



end
```

Figure 2.16: Assembly code for testbench3

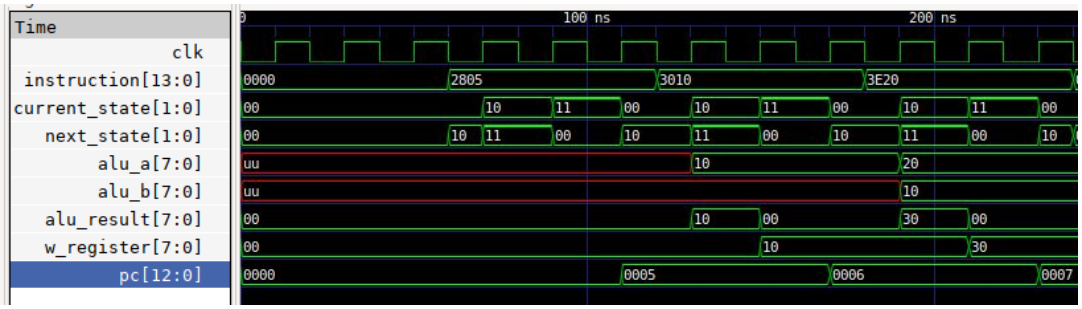

Figure 2.17: GOTO, MOVLW and ADDLW

- **Initial goto main**

The program begins with the standard initialization instruction that jumps to the main routine in program memory.

- **movlw 0x10**

This instruction demonstrates the basic move operation by loading the immediate value 10h into ALU input A during Execute stage. The alu_result becomes 10h and this value transfers to w_register in the subsequent cycle. This operation is functionally similar to movwf but without any memory access operations.

- **addlw 0x20**

The add-with-literal operation performs arithmetic between the W register and an immediate value. It reads the current W register value (10h) into alu_B while feeding the immediate value 20h to alu_A. The ALU computes the correct sum (30h) which is then stored back in w_register. This instruction shares similarities with addwf but operates exclusively with immediate data rather than memory contents.

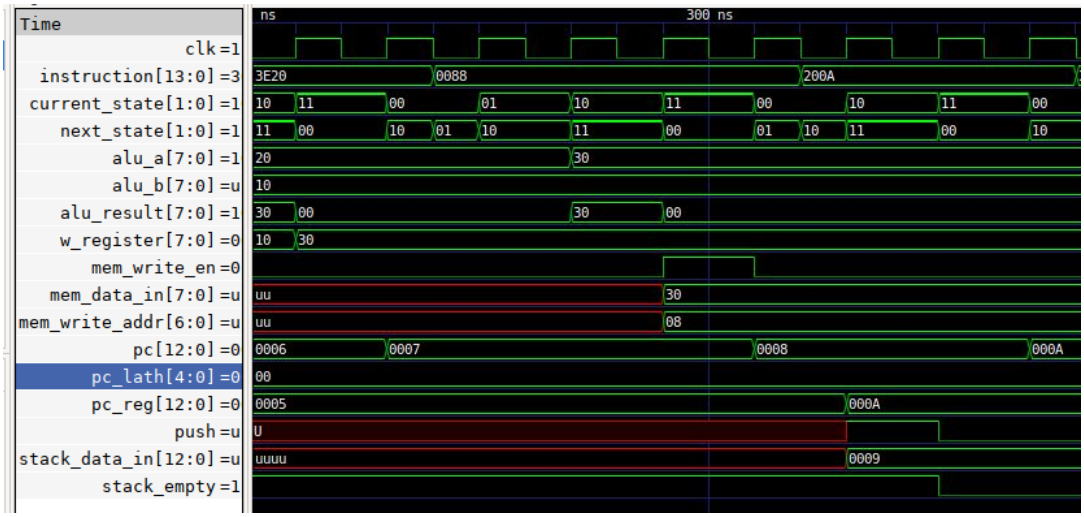

Figure 2.18: MOVWF and CALL math_operations

- **movwf COUNT1**

This instruction operation matches the behavior described in previous testbenches, successfully moving the W register value to the specified memory location COUNT1. The implementation details remain consistent with earlier observations.

- **call math_operations**

This subroutine call demonstrates proper program control flow management. The instruction executes in a single stage without requiring memory access for operands. It calculates the return address (pc + 1) for stack push operation while simultaneously updating pc_reg with the target address extracted from the instruction. The stack operation verification confirms correct behavior, showing empty status before operation and proper value storage after.

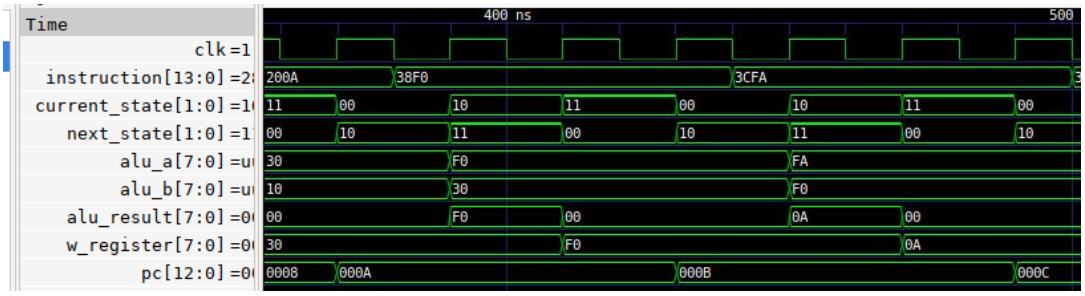

Figure 2.19: IORLW and SUBLW

- **iorlw 0xF0**

The inclusive OR operation combines the current W register value (30h) with the immediate operand F0h. The ALU correctly computes the result (F0h) which updates the W register. This demonstrates the bitwise logic capability of the processor with immediate operands.

- **sublw 0xFA**

This subtraction operation shows proper handling of literal arithmetic. The instruction computes FAh minus the current W register value (F0h), producing the correct result (0Ah) in the W register. The operation verifies the ALU's subtraction capability with immediate operands.

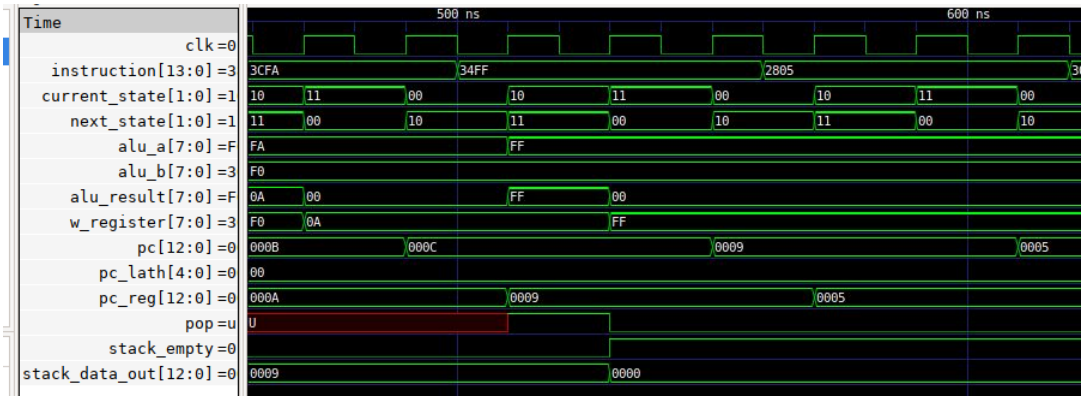

Figure 2.20: RETLW and GOTO main

- **retlw 0xFF**

The return-with-literal instruction demonstrates proper stack handling and program flow control. It reads the correct return address (09h) from the stack during Execute stage, visible through stack_data_out. The instruction updates pc_reg with this return address while simultaneously loading FFh into the W register. The subsequent instruction fetch confirms correct program flow by resuming at the instruction following the original call.

- **goto main**

This unconditional jump resets program flow to the main routine entry point at address 0x0005. The instruction properly updates the program counter, demonstrating the processor's ability to handle absolute jumps within the program memory space.
The test program creates an infinite loop by design. In the testbench environment, execution automatically terminates after 100 clock cycles through explicit testbench control, though the program logic itself would otherwise continue indefinitely.

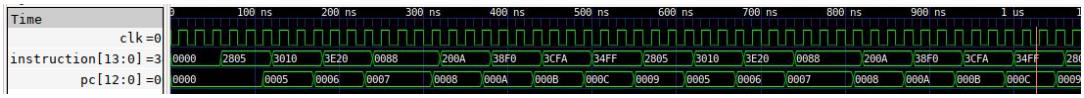

Figure 2.21: Infinite loop

Conclusion

These three testbenches showcases several exemples of a program's execution with the PIC microcontroller. Of course, these programs are not exhaustive since they do not test all inputs instructions we could execute with the PIC microcontroller. This would be too long to execute and analyse. We will now see if my code is synthesizable.

3 Synthesis

Initially, I executed the provided example to study its functionality before attempting synthesis with my own VHDL files. After adding the file paths to the configuration, I neglected to rerun the configuration script. Consequently, when I launched the batch mode synthesis flow, the tool persistently used the Verilog files instead of my VHDL designs.
Upon consulting with a teaching assistant, I rectified this oversight by rerunning the configuration script, which then properly recognized my files. During synthesis, I encountered a syntax error in one of my VHDL files where I had written:

```
elsif(status_write_enable)
```

instead of the correct:

```
elsif(status_write_enable = '1')
```

This mistake stemmed from my SystemVerilog background. Curiously, the GHDL compiler had not flagged this as an error during previous simulation runs.
After correcting the syntax and restarting the synthesis flow, the process completed successfully after considerable runtime. While I could examine the generated schematics, the reports revealed some inconsistencies.
With guidance from course assistants, I learned that the `constraints/timing_constraints.sdc` file needed modification as it was tailored specifically for the Verilog example. I implemented the following changes:

- Defined a clock signal connected to my design's `clk` port
- Incorporated the reset signal from my design
- Set the target clock frequency to 4MHz as specified in the PIC datasheet
- Removed the SCLK clock (from the Verilog example), maintaining only a single clock domain
- Eliminated all SCLK-related commands

I then configured the input and output constraints following the SPI module example.
During subsequent batch mode synthesis, I encountered formatting issues where commas were incorrectly interpreted as line terminators in the .sdc file. To resolve this, I migrated all commands to a new `timing_constraints.tcl` file. After rerunning synthesis successfully, I examined the `genus.log` file for errors, which only revealed warnings about potential unintentional latches - an issue I will address later after analyzing the reports.

```
77 # Inputs
78 set_input_delay [expr 0.1*${MASTER_CLK_P}] -clock DSP_MASTER [get_ports {
79     reset
80     instruction
81     pc_lath
82 }]
83 set_input_transition  1.0 [all_inputs]
84 set_driving_cell -lib_cell INVX4HVT  [all_inputs]
85
86
87 # Outputs
88 set_output_delay [expr 0.1*${MASTER_CLK_P}] -clock DSP_MASTER [get_ports {
89     pc
90     w_register
91     alu_A
92     alu_B
93     alu_result
94     alu_op
95     alu_bit_select
96     alu_status_in
97     alu_status_out
98     mem_write_addr
99     mem_read_addr
100    mem_data_in
101    mem_data_out
102    mem_write_en
103    mem_read_en
104    status_write_enable
105    push
106    pop
107    stack_data_in
108    stack_data_out
109 }]
110
111
```

Figure 3.1: timing constraints.tcl

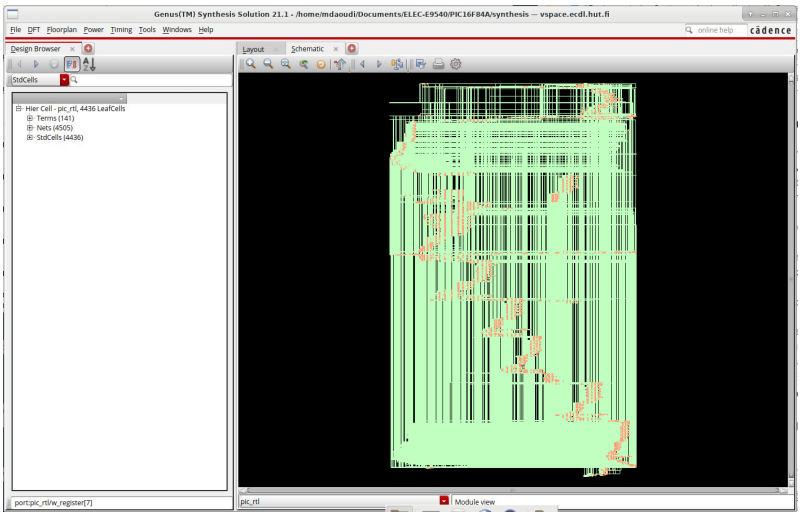

Figure 3.2: Schematic view.tcl

3.1 Reports Analysis

I will now systematically examine each report, though some require minimal commentary.

Type	CellArea	Percentage
datapath modules	0.00	0.00
external muxes	0.00	0.00
others	15461.48	100.00
total	15461.48	100.00

Figure 3.3: area.datapath.rpt

Instance Module	Cell Count	Cell Area	Net Area	Total Area
pic_rtl	4436	15461.478	5859.816	21321.294

Figure 3.4: area.summary.rpt

```
Summary
-------
------------------------------------------------------------------------
           Category           Number     %    Average Toggle Saving %
------------------------------------------------------------------------
 Total Clock Gating Instances        0  100.00                         -
------------------------------------------------------------------------
 RC Clock Gating Instances           0    0.00                      0.00
 Non-RC Clock Gating Instances       0    0.00                      0.00
------------------------------------------------------------------------
 RC Gated Flip-flops                 0    0.00                      0.00
 Non-RC Gated Flip-flops             0    0.00                      0.00
------------------------------------------------------------------------
 Total Gated Flip-flops              0    0.00                         -
 Total Ungated Flip-flops         1150  100.00                         -
 Enable not found                 1150  100.00                         -
------------------------------------------------------------------------
 Total Flip-flops                 1150  100.00                         -
------------------------------------------------------------------------
Multibit Flip-flop Summary
--------------------------
-------------------------------------------------------
Width     Number     Bits     RC Gated     Ungated
-------------------------------------------------------
1-bit     1150       1150     0 (0.00%)    1150 (100.00%)
-------------------------------------------------------
```

Figure 3.5: clock gating.rpt

NOR4X2HVT	1	3.420	0.043	0.161	slow_vdd1v0	0
OA21X1HVT	13	26.676	0.376	6.808	slow_vdd1v0	0
OA21X2HVT	1	3.078	0.043	1.662	slow_vdd1v0	0
OA21X4HVT	1	3.762	0.077	3.282	slow_vdd1v0	0
OA22X1HVT	27	64.638	0.814	13.424	slow_vdd1v0	0
OAI21X1HVT	16	27.360	0.461	7.228	slow_vdd1v0	0
OAI21X2HVT	1	3.078	0.038	0.884	slow_vdd1v0	0
OAI22X2HVT	6	20.520	0.264	2.812	slow_vdd1v0	0
OAI2BB1X1HVT	37	63.270	1.045	46.508	slow_vdd1v0	0
OAI2BB1X2HVT	4	12.312	0.180	4.943	slow_vdd1v0	0
OAI2BB2X2HVT	1	5.472	0.060	1.673	slow_vdd1v0	0
OAI31X1HVT	2	4.104	0.077	1.173	slow_vdd1v0	0
OAI31X4HVT	1	6.840	0.148	2.672	slow_vdd1v0	0
OR2X1HVT	40	54.720	1.063	19.921	slow_vdd1v0	0
OR2X2HVT	156	266.760	7.115	14.788	slow_vdd1v0	0
OR2X4HVT	17	46.512	1.605	10.307	slow_vdd1v0	0
OR3X1HVT	5	10.260	0.159	1.531	slow_vdd1v0	0
OR3X2HVT	3	7.182	0.146	1.196	slow_vdd1v0	0
OR3X4HVT	22	67.716	2.144	49.765	slow_vdd1v0	0
OR4X1HVT	19	38.988	0.874	7.574	slow_vdd1v0	0
OR4X6HVT	8	43.776	1.711	38.213	slow_vdd1v0	0
SDFFRHQX1HVT	104	853.632	13.087	1321.678	slow_vdd1v0	0
TLATNX1HVT	3	17.442	0.264	4.586	slow_vdd1v0	0
TLATNX2HVT	5	30.780	0.675	17.879	slow_vdd1v0	0
TLATNX4HVT	1	8.892	0.241	5.510	slow_vdd1v0	0
TLATX1HVT	106	580.032	9.155	183.397	slow_vdd1v0	0
TLATX2HVT	9	55.404	1.243	26.559	slow_vdd1v0	0
TLATX4HVT	21	172.368	5.188	126.947	slow_vdd1v0	0
XNOR2X1HVT	11	26.334	0.482	17.457	slow_vdd1v0	0
total	4436	15461.478	282.612	10477.175		

Type	Instances	Area	Area %	Leakage Power (nW)	Leakage Power %	Internal Power (nW)	Internal Power %
sequential	1295	7514.424	48.6	125.144	44.3	9183.326	87.7
inverter	120	145.692	0.9	3.767	1.3	48.456	0.5
buffer	28	145.692	0.9	5.093	1.8	183.313	1.7
logic	2993	7655.670	49.5	148.608	52.6	1062.080	10.1
physical_cells	0	0.000	0.0	0.000	0.0	0.000	0.0
total	4436	15461.478	100.0	282.612	100.0	10477.175	100.0

Figure 3.6: gates.rpt

The gates report provides detailed statistics about:

- Gate types and their instance counts
- Physical area occupation
- Power consumption and leakage characteristics
- Library references

Notably, the report indicates the presence of 145 latches (identified as TLAT... in the gate column). The summary section aggregates data by gate categories (sequential, inverter, buffer, etc.), revealing that sequential logic dominates power consumption at 87.7%, while combinational logic accounts for merely 10%.

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	1.08378e-07	8.81845e-06	8.80386e-07	9.80721e-06	66.61%
latch	1.67667e-08	3.64877e-07	3.41649e-07	7.23293e-07	4.91%
logic	1.57468e-07	1.29385e-06	2.74056e-06	4.19187e-06	28.47%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	2.82612e-07	1.04772e-05	3.96259e-06	1.47224e-05	99.99%
Percentage	1.92%	71.16%	26.92%	100.00%	100.00%

Figure 3.7: power.rpt

Regarding this alternate power report, I have no additional commentary except to note the anomalous absence of clock power consumption, which warrants further investigation.

```
Timing
--------

            Analysis view               Clock    Period
------------------------------------------------------------
func-slow_vdd1v0.setup_cworst_CCworst DSP_MASTER 250000.0


               Analysis                Cost   Critical        Violating
                 View                 Group  Path Slack  TNS    Paths
-------------------------------------------------------------------------------
func-slow_vdd1v0.setup_cworst_CCworst default   No paths  0.0          0
                                     in2out   No paths  0.0          0
                                     in2reg   220230.7  0.0          0
                                    reg2out   218079.0  0.0          0
                                    reg2reg   245802.4  0.0          0
-------------------------------------------------------------------------------
Total                                                    -0.0          0


Instance Count
--------------
Leaf Instance Count             4436
Physical Instance count            0
Sequential Instance Count       1295
Combinational Instance Count    3141
Hierarchical Instance Count        0

Area
----
Cell Area                          15461.478
Physical Cell Area                 0.000
Total Cell Area (Cell+Physical)    15461.478
Net Area                           5859.816
Total Area (Cell+Physical+Net)     21321.294

Max Fanout                         1150 (clk)
Min Fanout                         1 (alu_status_out_reg[0])
Average Fanout                     2.8
Terms to net ratio                 3.8708
Terms to instance ratio            3.9310
Runtime                            207.8598060000001 seconds
Elapsed Runtime                    205 seconds
Genus peak memory usage            1273.22
Innovus peak memory usage          no_value
Hostname                           vspace.ecdl.hut.fi
```

Figure 3.8: qor.rpt

Key observations from timing analysis:

1. **Timing Closure**: Achieved with zero violations (TNS = 0.0) across all path groups (in2reg, reg2out, reg2reg)
2. **Slack Analysis**: Substantial margins observed (e.g., 220230.7ps for in2reg) suggesting the 250ns clock period (4kHz) could be reduced for performance optimization
3. **Area Metrics**:
   - Cell Area: 15,461.48 units
   - Net Area: 5,859.82 units
   - Total: 21,321.29 units with a 38% net/cell ratio indicating reasonable routing
4. **Instance Statistics**:
   - 1,295 sequential cells (29.2% of total) - appropriate for microcontroller designs
   - Maximum fanout of 1,150 on `clk`, properly buffered without skew issues

```
Path 1: MET (218079 ps) Late External Delay Assertion at pin alu_status_out[2]
          View: func-slow_vdd1v0.setup_cworst_CCworst
         Group: reg2out
    Startpoint: (R) current_state_reg[0]/CK
         Clock: (R) DSP_MASTER
      Endpoint: (R) alu_status_out[2]
         Clock: (R) DSP_MASTER

                   Capture       Launch
       Clock Edge:+  250000            0
      Src Latency:+       0            0
      Net Latency:+    1000 (I)     1000 (I)
          Arrival:=  251000         1000

     Output Delay:-   25000
      Uncertainty:-     100
    Required Time:=  225900
     Launch Clock:-    1000
        Data Path:-    6821
            Slack:=  218079

Exceptions/Constraints:
  output_delay            25000           timing_constraints.t_line_88_74_1

#-------------------------------------------------------------------------------------------------------
#      Timing Point       Flags   Arc   Edge     Cell      Fanout Load Trans Delay Arrival Instance
#                                                             (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------------------
  current_state_reg[0]/CK -       -     R     (arrival)      1150    -     0     0    1000    (-,-)
  current_state_reg[0]/Q  -       CK->Q R     DFFRX1HVT         4  4.8   164   479    1479    (-,-)
  g102725/Y               -       A->Y  R     OR2X1HVT          3  4.4   142   264    1743    (-,-)
  g102689/Y               -       A->Y  R     OR2X2HVT          5  6.5   122   272    2015    (-,-)
  g102674/Y               -       A->Y  F     INVX1HVT          2  2.8   162   172    2187    (-,-)
  g102299/Y               -       A->Y  F     AND2X1HVT         4  4.9   241   264    2451    (-,-)
  g102276/Y               -       AN->Y F     NOR2BX1HVT        1  2.2   144   244    2695    (-,-)
  g102270/Y               -       B->Y  R     NOR2X2HVT         3  5.0   202   199    2893    (-,-)
  g101531/Y               -       B->Y  R     OR2X4HVT         11 12.3   125   295    3188    (-,-)
  g101509/Y               -       A->Y  F     INVX2HVT          5  6.3   185   177    3366    (-,-)
  g101235/Y               -       B->Y  F     AND2X2HVT         7  7.9   208   282    3647    (-,-)
  g100925/Y               -       A->Y  F     AND2X1HVT         1  1.8   102   214    3861    (-,-)
  g100776/Y               -       C->Y  F     OR3X1HVT          2  2.9   160   292    4153    (-,-)
  g100623/Y               -       B->Y  R     NOR2X1HVT         1  1.8   152   186    4339    (-,-)
  g100573/Y               -       B1->Y R     OA22X1HVT         1  1.9    84   338    4677    (-,-)
  g100556/Y               -       B0->Y F     OAI2BB1X1HVT      1  1.9   251   230    4907    (-,-)
  g100538/Y               -       B0->Y R     AOI21X1HVT        1  1.8   167   252    5159    (-,-)
  g100518/Y               -       B0->Y R     OA21X1HVT         1  1.9    81   325    5483    (-,-)
  g100492/Y               -       B->Y  F     NAND2X1HVT        1  1.9   249   206    5690    (-,-)
  g100485/Y               -       B->Y  R     NOR2X1HVT         1  2.3   190   258    5948    (-,-)
  g98865__6783/Y          -       B->Y  F     NAND2X2HVT        1  2.2   188   227    6175    (-,-)
  g98800__7482/Y          -       B->Y  R     NOR2X2HVT         2  3.9   175   204    6379    (-,-)
  g98799/Y                -       A->Y  F     CLKINVX4HVT       2 10.6   260   240    6619    (-,-)
```

Figure 3.9: Extract from timing.rpt

The timing report confirms all constraints are satisfied with no path violations.

3.2 Conclusion

Potential future work includes:

- Frequency scaling experiments to determine optimal performance
- Area and power consumption optimization

I would have to determine how the area and power scale with the frequency and run several simulations with different frequency lower then 4 Mhz. Unfortunately, I have other projects to work on and for the moment, I can not run these simulations. But if I have time, I will describe them after this part. If there's nothing more then it means I didn't have time.
About the latches, I know where the problem comes from in my code. For instances, in my rtl code I assign a value to w_register when the state is Mwrite but I don't assign any other value to w_register when I'm not in this state. So what the compiler (ghdl) understands is that w_register should keep it's value until the state is Mwrite again. This is creating a latch. What I should do for all these signals is always assign them inside the process. A solution that comes to mind right now is to do everytime the process launches:
w_register <= w_register
signal_that_should_keep_its_value <= signal_that_should_keep_its_value

Unfortunately, this implies to modify my code. It may be a simple modification but I just don't know what will happen. So I won't modify for the moment because I don't have the necessary time right now. I'm writing this t show that I know what is the error even though I'm not correcting it.

Cocnlusion

I think I can say that I successfully implemented a functional PIC16F84A microcontroller replica in VHDL. It was very interesting to see the different steps and do the whol process from the start by writing ourselves all the submodules used in the top rtl code. Having not too much specifications was also agood idea because we could do anything we wanted in term of design as long as it works at the end.
It really helped me understand how a processor actually works on a lower level, and how all the pieces fit together—from the ALU to the memory handling to the control unit. Writing everything ourselves gave me a better appreciation of how much goes into even a simple microcontroller. Of course, there were a few challenges along the way, especially debugging and making sure all the timing was correct, but in the end, it was satisfying to see it running properly. Overall, I learned a lot through this project, not just about VHDL and digital design, but also about how to structure a project like this and approach problems step by step.